

Project context: Sanctuary — multi-user diary/wellness app with cloud file sync

What the app is: Sanctuary is a single-page HTML/CSS/JS app (index.html) — a personal diary/wellness tracker with a dashboard, diary entries, a notebook with file attachments, daily goals, projects, a "life journey" timeline, a photo gallery, a favourite-places list with photos, an analytics view, break activities, a music player with playlists, an alarm/reminder "watch" screen, a memory game, and an "all about you" profile section. It's designed to be installed as a PWA and used by many different people (not just one person), so every user's data and files must be private to them and sync across their own devices.

Backend architecture, as it stands now: Supabase handles authentication (email/password sign-up and sign-in) and stores all of the user's *text/structured data* — goals, notes, diary entries, projects, achievements, milestones, alarms, reminders, trash, AAY profile fields, etc. — in a single table called `sanctuary_data`, one row per user, protected by Postgres Row Level Security policies so each user can only read/write their own row. There's also a small profiles table holding just the avatar URL. The SQL to create both tables and their RLS policies lives in a file called `supabase_setup.sql`, which is idempotent (safe to re-run). This part of the stack was never changed during the storage migration and is working correctly.

Why file storage moved away from Supabase: Supabase's free tier only gives 1GB of file storage and a 5GB/month bandwidth cap, which is too small for a multi-user app storing photos, music, and documents (PDF/Word/Excel). We evaluated Cloudflare R2 (10GB free storage, zero egress fees forever) as the obvious upgrade, but R2 requires a credit card or PayPal on file to activate even on the free tier, which the user doesn't have. We switched the plan to **Backblaze B2** instead — also 10GB free storage, free egress up to 3x stored volume, and critically, **no credit card required to sign up or use the free tier**. B2 exposes an S3-compatible API.

Why a Cloudflare Worker sits in the middle: B2's S3-compatible API requires AWS Signature Version 4 authentication with a secret Application Key. That secret can never be embedded in client-side JavaScript (unlike Supabase's anon key, which is safe to expose because RLS enforces security server-side) — anyone could read it from the browser and wipe out every user's files. So we built a small Cloudflare Worker (free tier, no card required — that requirement is specific to R2, not Workers) that acts as a secure proxy: the browser app sends files to the Worker, the Worker holds the B2 credentials as encrypted secrets and signs the actual S3 requests using the `aws4fetch` npm library (chosen because Cloudflare Workers don't have a native B2 binding the way they do for R2). The Worker also enforces per-user folder isolation — every file is stored at a path like `<supabase_user_id>/<filename>`, so one user's files are never reachable using another user's ID.

The Worker's API surface: Three routes — `POST /upload/<userId>/<filename>` (uploads a file, body is raw bytes), `GET /file/<userId>/<filename>` (downloads/streams a file), and `DELETE /file/<userId>/<filename>` (deletes a file). Upload and delete require an `Authorization: Bearer <APP_SECRET>` header, where `APP_SECRET` is a long random string the user generated themselves and stored both as a Worker secret (wrangler secret put `APP_SECRET`) and as a constant in the app's JS (`STORAGE_APP_SECRET`). GET requests are deliberately **not** auth-checked, because browser `<img>`, `<audio>`, and `<iframe>` tags cannot attach custom headers — this was the cause of a real bug we hit and fixed (see below).

Filenames are long random strings, so an unauthenticated-but-unguessable GET is an acceptable privacy tradeoff for this use case.

Worker secrets configured via Wrangler CLI: APP_SECRET (shared password between app and Worker), B2_KEY_ID and B2_APP_KEY (the Backblaze Application Key pair, created via Backblaze's dashboard under Application Keys, scoped to one bucket), B2_BUCKET (the bucket name, e.g. sanctuary-files), and B2_ENDPOINT (the full URL form of the bucket's S3 endpoint, e.g. <https://s3.us-east-005.backblazeb2.com> — note it must include the <https://> scheme even though Backblaze's dashboard displays it without one). The user successfully deployed this Worker; it's live at <https://sanctuary-storage.myapp2026.workers.dev>. The Backblaze bucket itself was created as Private, with Default Encryption enabled (Backblaze-managed SSE-B2) and Object Lock disabled (Object Lock would have prevented the app from ever deleting/replacing files, which would break core functionality).

App-side code changes made in index.html: Added two config constants near the Supabase client setup — STORAGE_WORKER_URL (the deployed Worker's URL) and STORAGE_APP_SECRET (matching the Worker's APP_SECRET) — plus two helper functions, storageUpload(file, keyName) and storageDelete(keyName), that wrap fetch calls to the Worker. Every place in the app that used to upload directly to Supabase Storage buckets (avatar photo, note attachments in the Notebook, gallery/vision-board photos, favourite-place photos, and music tracks) was rewritten to call storageUpload() instead, and each of those items now also stores a storageKey field (the exact filename used on the Worker/B2 side) so it can be deleted precisely later rather than guessed at. The IndexedDB-based local caching layer (for instant playback/display without a network round-trip) was kept intact and made to gracefully fall back to the cloud URL when the local cache is empty, e.g. on a different device. Trash/permanent-delete logic was extended to also call storageDelete() using the stored storageKey, so permanently deleted items actually get removed from the B2 bucket instead of silently accumulating there forever. The syncToSupabase()/loadFromSupabase() functions (which sync the text-data table) were updated to carry storageUrl and storageKey fields through so cross-device loads display photos immediately via the cloud URL rather than waiting on a local blob that won't exist on a new device.

Bugs found and fixed after initial deployment: (1) Viewing/playing previously-uploaded files returned 401 Unauthorized — root cause was the Worker requiring the Authorization header on GET requests too, which browser `<img src>/<audio src>` tags can never send; fixed by removing the auth check specifically for GET /file/... while keeping it for upload/delete. (2) Saving a new Favourite Place appeared to silently fail — root cause was actually a JS crash *after* a successful save, inside renderPlaces(), where code assumed a photo object always had a .url string and called .replace() on it; if ph.url was null/undefined (which happened transiently because of bug #1 happening concurrently), .replace() was called on a non-string and threw, aborting the re-render and making it look like the save itself had failed. Fixed by adding a safe string-extraction guard in both renderPlaces() and renderPlacePhotos(). (3) Clicking a PDF attachment in the Notebook downloaded the file instead of opening it in the app's in-page viewer (the iframe-based attachment viewer modal) — root cause was the Worker's GET handler never setting a Content-Disposition header, so the browser used B2's default (often attachment), forcing a download regardless of being

loaded in an iframe; fixed by having the Worker explicitly set Content-Disposition: inline by default, with an opt-in ?download=1 query parameter that switches it to attachment — and the app's existing Download button in the viewer modal was updated to append ?download=1 to its link so it still forces a real download rather than relying on the browser's download attribute alone.

Current state: working. Music upload/playback, photo gallery upload/display, favourite-places photos, and note attachments (including PDF inline preview) have all been confirmed, by the user, to be functioning correctly across the Worker + B2 + Supabase stack, including files that were uploaded before the bugs were fixed (they didn't need re-uploading — fixing the Worker retroactively made them accessible).

Files involved, for reference if continuing this work: index.html (the full app), supabase_setup.sql (DB schema + RLS, run once in Supabase's SQL editor), and the sanctuary-worker-b2/ project folder containing wrangler.toml (Worker config, no R2 binding needed since this uses B2 not R2), package.json (declares the aws4fetch dependency), and src/index.js (the actual Worker code implementing the upload/file/delete routes described above). Deployment is via npx wrangler deploy from inside that folder, and any future code changes to the Worker require re-running that command for them to take effect — editing the local file alone does nothing until redeployed.